

## Section 1: Attack summary (5 marks)

1. **Q1:** Which tool was used to simulate a vulnerable web server in the penetration test? (MCQ, 1 mark)

PHP built-in server

2. **Q2:** What command created a new user for simulating a vulnerable service? (MCQ, 1 mark)

`sudo adduser webuser --disabled-password --gecos ""`

3. **Q3:** Describe the purpose of creating a separate 'webuser' and how it supports ethical testing. (Short answer, 3 marks)

### Purpose of Creating a Separate 'webuser' Account

Creating a separate webuser account is a critical step in simulating a realistic and ethically sound penetration testing environment. The primary goal is to mimic real-world system configurations where services such as web servers run under dedicated, unprivileged users rather than root. This setup supports responsible testing by limiting potential damage during exploitation and aligning with actual security practices found in production systems.

The technical rationale is that services like Apache or Nginx are best run under non-root, limited-permission users to reduce the attack surface. By assigning a specific account (like webuser) to run a web service, testers simulate a hardened configuration while also providing an identifiable and controllable target for privilege escalation testing.

The use of webuser supports ethical testing by clearly isolating the test environment from critical system components. This ensures that exploits are directed only at a designated target account, preventing unintentional system-wide impact and adhering to legal and professional standards of controlled testing. It allows for a safe, reversible, and scoped engagement, essential in ethical hacking.

The response reflects a strong awareness of simulation realism. Using a separate webuser introduces role separation, which is a key principle in system hardening and real-world operations. It simulates the common security practice where services are sandboxed under limited users — helping emulate authentic attack paths (like local file inclusion, privilege escalation, or directory traversal) and testing how well defenses handle these boundaries.

## Section 2: Vulnerability report – Web upload (4 marks)

1. **Q4:** What was the primary goal of placing upload.php in the simulated web server? (MCQ, 1 mark)

Provide a remote command execution backdoor

2. **Q5:** What file contained the simulated web flag? (MCQ, 1 mark)

flag.txt

3. **Q6:** What is one way to prevent attackers from gaining remote shell access through a malicious file upload? (Short answer, 2 marks)

One effective way to prevent attackers from gaining remote shell access through a malicious file upload is to:

Store uploaded files outside the web root directory (e.g., /var/www/uploads → /var/uploads)

By storing uploads outside the web root, even if a .php, .jsp, or .asp file is uploaded, it cannot be directly executed via HTTP because the server does not serve files from that directory.

This mitigation prevents web shells from being triggered through a browser or HTTP request. Even if an attacker uploads a script

## Section 3: Network fuzzing findings (5 marks)

1. **Q7:** In the fuzzing phase, what did the fake echo service return in response to input? (MCQ, 1 mark)

"ECHO: [user input]"

2. **Q8:** What is a potential security concern with exposing the echo service as implemented? (MCQ, 1 mark)

Command injection or reconnaissance abuse

3. **Q9:** Select all activities that were part of your post-exploitation testing. (Multi-select, 3 marks – 1 mark per correct answer)

Configuring persistence with cron

Uploading a reverse shell

Harvesting credentials

#### Section 4: Metasploit usage (5 marks)

1. **Q10:** What type of payload was generated with msfvenom during the exploitation step? (MCQ, 1 mark)  
  
PHP Meterpreter reverse shell
2. **Q11:** Which Metasploit module was used to handle the reverse shell connection? (MCQ, 1 mark)  
  
exploit/multi/handler
3. **Q12:** List two risks associated with leaving a reverse shell handler active after testing. (Short answer, 3 marks)

Two Risks Associated with Leaving a Reverse Shell Handler Active After Testing:

##### **Unauthorized Access or Security Exposure:**

Leaving a reverse shell handler active after testing can expose the system to unauthorized access. If the handler is still listening, an attacker could potentially exploit the open connection or find a way to interact with the reverse shell, which may allow them to gain control of the target system. This could lead to a security breach if the system is compromised after the testing phase is completed.

##### **Violation of Professional or Procedural Standards:**

Leaving a reverse shell handler running after testing is considered bad practice and violates professional standards for cybersecurity. It's crucial to properly clean up after penetration tests or security assessments. Failing to remove the handler could be seen as negligence, and it may be viewed as a procedural failure that could damage the reputation of the tester or the organization conducting the test.

#### Section 5: CVSS evaluation (3 marks)

1. **Q13:** Which CVSS vector string best describes a remote file upload that grants full system control with no user interaction? (MCQ, 1 mark)  
  
AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
2. **Q14:** Justify why this CVSS vector is considered critical in the context of this lab. (Short answer, 2 marks)

The CVSS vector in question — AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H — represents a critical vulnerability because it signifies an exploit with significant security risks that are remotely exploitable, easy to execute, and can have devastating consequences on the target system.

## Section 6: Persistence reflection (3 marks)

1. **Q15:** According to the lab, where was the cron-based persistence payload configured? (MCQ, 1 mark)

User crontab

2. **Q16:** Define a stealthier persistence technique than cron jobs and explain its potential impact. (Short answer, 2 marks)

### Stealthier Persistence Technique: Rootkits with Library Hooking

One of the stealthier persistence techniques than cron jobs is using rootkits with library hooking. This method involves injecting malicious code into system libraries or hooking into critical system functions, allowing the attacker to maintain persistence on the system while remaining largely undetected by traditional monitoring methods.

#### Explanation of the Technique:

**Library Hooking:** The attacker can insert their malicious code into shared libraries or manipulate system calls. By hooking into these libraries, such as libc, the attacker can redirect calls to malicious functions, ensuring that their payload is executed whenever the affected library or system function is called.

**Rootkits:** A rootkit is a set of tools that hides the existence of malicious software by modifying system processes, files, and logs. The rootkit can intercept system calls, hide files and network connections, and generally allow the attacker to operate undetected for a long period of time. These rootkits often reside in user-space or kernel-space and can be used to hook into various system operations.

#### Why This is Stealthier:

**Bypassing Cron Detection:** Unlike cron jobs, which are visible to system administrators via cron configuration files or logs, library hooking works at a much deeper level, often at the API or system call level, making it harder to detect using traditional monitoring tools.

**Hidden in Plain Sight:** The attacker's code could be executed as part of regular system operations (such as when system libraries are loaded or certain system calls are made), meaning it is less likely to be flagged as unusual by routine monitoring processes.

**Persistence Across Reboots:** This technique is often more persistent than cron jobs because it does not depend on external cron files or user crontab entries. The malicious code can remain undetected as part of the system's core functions.

**Potential Impact:**

**Elevated Access:** Rootkits can grant attackers full control of the compromised system, including the ability to hide their presence from system administrators and security tools.

**Data Exfiltration and Modification:** Attackers can stealthily monitor and modify sensitive data or steal information without alerting the user or security tools.

**Long-Term Persistence:** Since the rootkit operates at a low level, it is much harder to remove and detect. Even if the attacker's original entry point is patched, the rootkit can remain active and restore access.

**System Integrity:** The presence of a rootkit can lead to a significant compromise of system integrity, allowing attackers to manipulate critical system functions without detection.

**Section 7: Recommendations (4 marks)**

1. **Q17:** Which of the following actions would be most effective in preventing reverse shell execution from web uploads? *(MCQ, 1 mark)*

Disable file uploads on public-facing forms

2. **Q18:** What recommendation would you make to improve the detection of such attacks within an enterprise? *(Short answer, 3 marks)*

**Recommendation to Improve Detection of Reverse Shell Attacks from Web Uploads:**

Implement Web Application Firewalls (WAFs) with File Upload Scanning and Enable Intrusion Detection Systems (IDS)

Explanation:

To improve detection of reverse shell attacks stemming from web uploads, enterprises should implement Web Application Firewalls (WAFs) specifically tuned to detect malicious file uploads and intrusion detection systems (IDS) that monitor suspicious behavior on the network and system levels. These solutions can analyze incoming traffic and files in real-time, identify malicious patterns or exploits, and alert administrators.

### **Section 8: Risk rating and final thoughts (11 marks)**

1. **Q19:** What is the main reason post-exploitation cleanup is recommended in professional pentests? *(MCQ, 1 mark)*  
  
To simulate attacker behavior and cover tracks
2. **Q20:** Upload a 1-paragraph executive summary of your penetration test findings. Include scope, main exploit used, and key risk. *(File upload, 10 marks – based on 5-point rubric)*

**Separate PDF file to be attached**